

# NiceGUI `testing.screen_plugin` 插件详细解析

`nicegui.testing.screen_plugin` 是 NiceGUI 官方自动化测试体系中**面向 UI 视觉验证与全局状态检测**的核心插件，与 `user_plugin` 侧重“模拟用户交互”不同，`screen_plugin` 聚焦于“全局视角的 UI 状态断言、视觉对比、页面结构验证”，基于 Playwright 的截图 / 定位能力封装，提供简洁的 API 来验证页面整体状态、组件布局、视觉一致性，是 NiceGUI 应用 UI 测试的重要补充。

## 一、核心定位与设计初衷

在 NiceGUI 测试场景中，`user_plugin` 解决了“用户交互模拟”的问题，但仅靠交互断言（如“按钮点击后文本是否变化”）无法覆盖以下需求：

- 验证组件的**视觉呈现**（如颜色、尺寸、位置是否符合预期）；
- 全局检测页面上的组件集合（如“是否存在 3 个按钮”“表单是否完整渲染”）；
- 跨平台 / 浏览器的视觉一致性验证（如截图对比）；
- 无需定位单个组件，直接断言“页面是否包含指定文本 / 元素类型”。

`screen_plugin` 针对上述痛点设计：

- 提供全局视角的断言 API（如 `assert_text`、`assert_no_text`）；
- 内置视觉对比能力（截图与基准图比对）；
- 简化页面结构验证（如统计组件数量、验证组件可见性）；
- 与 `user_plugin` 完全兼容，可组合使用（交互 + 全局验证）。

## 二、前置准备

### 1. 依赖安装

与 `user_plugin` 共享基础依赖，只需确保安装完整的测试依赖：

```
# 安装 NiceGUI 测试套件（含 screen_plugin）
pip install nicegui[test]

# 初始化 Playwright 浏览器驱动（首次使用）
playwright install
```

### 2. 核心概念

`screen_plugin` 的核心对象是 `Screen`，它封装了 Playwright 的 `Page` 对象，提供全局 UI 检测能力：

- 无需手动定位单个组件，直接对整个页面做断言；
- 内置“等待元素出现 / 消失”的逻辑，避免异步渲染导致的断言失败；
- 支持截图、视觉对比等视觉验证能力。

## 三、核心使用流程

### 1. 基础示例（应用源码 `app.py`）

```
# app.py
from nicegui import ui

def main():
    # 构建一个简单的表单页面
    with ui.card():
        ui.label("用户注册").style("font-size: 20px; color: #2196F3;")
        ui.input(label="用户名", placeholder="请输入用户名")
        ui.input(label="密码", password=True)
        ui.button("注册", color="primary")
        ui.label("已有账号? ").style("color: #666;")
        ui.link("登录", "/login")

    ui.run(native=False, show=False, port=8080) # 测试专用启动配置

if __name__ == "__main__":
    main()
```

### 2. 测试用例（`test_app.py`）

```
# test_app.py
import pytest
from nicegui import testing
from nicegui.testing import screen_plugin
from app import main

# 启动应用并注入 screen_plugin
@pytest.fixture(scope="module")
def screen():
    # 启动应用，绑定 Screen 对象（全局视角）
    with testing.run_app(main) as page:
        # 初始化 Screen 插件
        screen = screen_plugin.Screen(page)
        yield screen

# 核心测试用例：全局 UI 断言
def test_screen_basic(screen: screen_plugin.Screen):
    # 1. 断言页面包含指定文本（全局搜索）
    screen.assert_text("用户注册")
    screen.assert_text("用户名")
    screen.assert_text("注册")
    screen.assert_text("登录")

    # 2. 断言页面不包含错误文本
    screen.assert_no_text("错误")
    screen.assert_no_text("服务器异常")

    # 3. 断言元素可见性（通过文本定位）
```

```
assert screen.is_visible("用户注册")
assert screen.is_visible("注册")

# 4. 等待元素出现（处理异步渲染）
screen.wait_for_text("已有账号?", timeout=3000)

# 5. 统计指定文本的元素数量
assert screen.count_text("输入") == 2 # 两个输入框的“输入”文本
```

3. 运行测试

```
pytest test_app.py -v
```

四、核心 API 详解

screen\_plugin.Screen 的核心方法分为**文本断言**、**视觉验证**、**元素检测**、**辅助工具**四类，具体如下：

| 类别   | 方法                                                     | 作用                                    | 示例                                             |
|------|--------------------------------------------------------|---------------------------------------|------------------------------------------------|
| 文本断言 | assert_text(text, timeout=5000)                        | 断言页面包含指定文本，超时抛出异常                     | screen.assert_text("提交")                       |
|      | assert_no_text(text, timeout=5000)                     | 断言页面 <b>不包含</b> 指定文本                  | screen.assert_no_text("失败")                    |
| 视觉验证 | screenshot(path, full_page=True)                       | 截取页面截图（全屏 / 可视区域）                     | screen.screenshot("screenshots/current.png")   |
|      | assert_screenshot_match(baseline_path, threshold=0.01) | 截图与基准图比对（误差阈值）                        | screen.assert_screenshot_match("baseline.png") |
| 元素检测 | is_visible(text/locator)                               | 检测元素是否可见（支持文本 / Playwright 定位器）       | screen.is_visible("登录按钮")                      |
|      | count_text(text)                                       | 统计页面中包含指定文本的元素数量                      | screen.count_text("选项")                        |
|      | wait_for_text(text, timeout=5000)                      | 等待指定文本出现，超时抛出异常                       | screen.wait_for_text("加载完成")                   |
|      | wait_for_no_text(text, timeout=5000)                   | 等待指定文本消失                              | screen.wait_for_no_text("加载中")                 |
| 辅助工具 | get_by_text(text)                                      | 基于文本获取 Playwright 定位器（兼容 user_plugin） | locator = screen.get_by_text("提交")             |
|      | page                                                   | 暴露底层 Playwright Page 对象（扩展使用）         | screen.page.locator("input").count()           |

五、进阶使用场景

1. 视觉对比测试（截图验证）

适用于验证 UI 视觉一致性（如组件样式、布局是否符合设计），需先生成基准截图，再对比测试：

```
def test_visual_consistency(screen: screen_plugin.Screen):
    # 步骤1: 生成基准截图（首次运行时执行，后续注释）
    # screen.screenshot("baseline/register_form.png", full_page=True)

    # 步骤2: 对比当前截图与基准截图（误差阈值0.01，即1%）
    screen.assert_screenshot_match(
        baseline_path="baseline/register_form.png",
        threshold=0.01,
        full_page=True
    )
```

**注意：**视觉对比对浏览器版本、分辨率敏感，建议在固定环境（如 CI 容器）中运行。

## 2. 结合 `user_plugin` 实现“交互 + 全局验证”

`screen_plugin` 可与 `user_plugin` 组合使用，先模拟用户交互，再全局验证 UI 状态：

```
from nicegui.testing import user_plugin, screen_plugin

@pytest.fixture(scope="module")
def test_env():
    with testing.run_app(main) as page:
        user = user_plugin.Page(page)
        screen = screen_plugin.Screen(page)
        yield {"user": user, "screen": screen}

def test_interact_and_verify(test_env):
    user = test_env["user"]
    screen = test_env["screen"]

    # 1. user_plugin 模拟输入
    input_box = user.get_by_label("用户名")
    user.type(input_box, "test_user")

    # 2. screen_plugin 全局验证输入结果
    screen.assert_text("test_user") # 断言页面包含输入的用户名

    # 3. 模拟点击注册按钮
    user.click(user.get_by_text("注册"))

    # 4. 验证加载状态消失
    screen.wait_for_no_text("加载中")
```

## 3. 验证动态加载的内容

针对异步渲染（如后台任务完成后显示的内容），使用 `wait_for_text` 确保断言时机正确：

```
# app.py 新增动态内容逻辑
from nicegui import ui, background_tasks
import asyncio

async def load_data():
```

```
        await asyncio.sleep(2)
        ui.label("数据加载完成").style("color: green;")

ui.button("加载数据", on_click=lambda: background_tasks.create(load_data()))

# test_app.py 测试用例
def test_dynamic_content(screen: screen_plugin.Screen):
    # 模拟点击按钮（结合 user_plugin）
    user = user_plugin.Page(screen.page)
    user.click(user.get_by_text("加载数据"))

    # 等待动态内容出现
    screen.wait_for_text("数据加载完成", timeout=3000)
    # 断言内容可见
    assert screen.is_visible("数据加载完成")
```

六、关键注意事项

1. 文本唯一性

`assert_text` / `count_text` 等方法基于文本全局匹配，若页面存在重复文本（如多个“提交”按钮），需结合 `user_plugin` 的精准定位或 Playwright 原生 locator 补充验证。

2. 视觉对比的局限性

- 对动态内容（如时间戳、随机数）敏感，需排除动态区域（可通过 `mask` 参数遮罩）；
- 不同操作系统的字体、渲染差异可能导致误判，建议仅对核心静态布局做视觉验证。

3. 超时配置

所有异步等待方法（如 `wait_for_text`）默认超时 5 秒，针对慢加载场景（如后台任务），需手动调整 `timeout` 参数（单位：毫秒）。

4. 原生组件限制

无法验证 `ui.native` 原生组件（如原生窗口、系统对话框），这类组件需单独通过系统级测试工具（如 PyAutoGUI）验证。

七、与 `user_plugin` 的核心差异

| 特性   | <code>screen_plugin</code>            | <code>user_plugin</code>               |
|------|---------------------------------------|----------------------------------------|
| 核心视角 | 全局页面视角（无差别检测所有元素）                     | 组件视角（精准定位 / 交互单个组件）                    |
| 核心能力 | 全局断言、视觉验证、数量统计                        | 模拟用户交互（点击 / 输入 / 选择）                   |
| 定位方式 | 基于文本 / 截图（无需关联组件对象）                   | 基于组件对象 <code>/label/text</code> （精准定位） |
| 典型场景 | 验证页面整体状态、视觉一致性、文本存在性                  | 模拟用户操作、测试组件交互逻辑                        |
| 依赖关系 | 可独立使用，也可与 <code>user_plugin</code> 组合 | 核心用于交互，需结合断言工具验证结果                     |